Mining Association Rules in Large Databases

Association Rules in a nutshell

- Frequent patterns (or frequent itemsets)
  - What items are frequently purchased together in your Walmart?
- Association, correlation vs. causality
  - A typical association rule
    - Bread → Butter [5%, 75%] (support, confidence)

Lecture outline

- **Task 1: Methods for finding all frequent itemsets efficiently**

- Task 2: Methods for finding association rules efficiently

Association rules / Association Rule Mining

- Given a set of transactions **D**, find rules that will predict the occurrence of an item (or a set of items) based on the occurrences of other items in the transaction
- Unsupervised Learning

Market-Basket transactions

TID	*Items*
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

Examples of association rules

{Cookies} $\rightarrow$ {Butter},
{Milk, Bread} $\rightarrow$ {Cookies, Coke},
{Butter, Bread} $\rightarrow$ {Milk},

An even simpler concept: frequent itemsets

- Given a set of transactions **D**, find combination of items that occur frequently

Market-Basket transactions

TID	*Items*
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

Examples of frequent itemsets

{Cookies, Butter},
{Milk, Bread}
{Butter, Bread, Milk},

Definition: Frequent Itemset

- **Itemset**
  - A set of one or more items
    - E.g.: {Milk, Bread, Cookies}
  - k-itemset
    - An itemset that contains k items
- **Support count (σ)**
  - Frequency of occurrence of an itemset (number of transactions it appears)
  - E.g. **σ({Milk, Bread, Cookies}) = 2**
- **Support**
  - Fraction of the transactions in which an itemset appears
  - E.g. **S({Milk, Bread, Cookies}) = 2/5**
- **Frequent Itemset**
  - An itemset whose support is greater than or equal to a ***minsup*** threshold

TID	*Items*
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

Why do we want to find frequent itemsets?

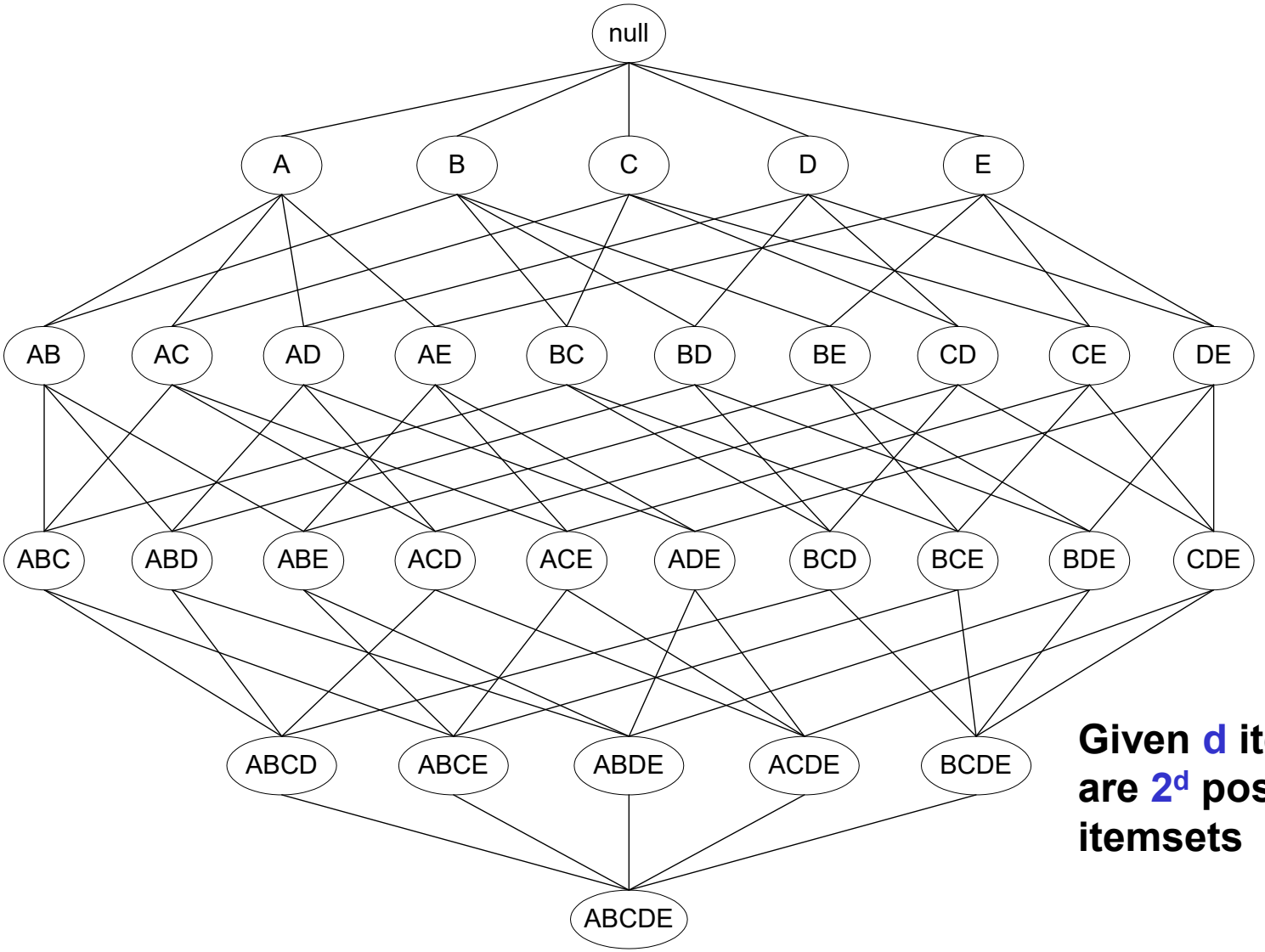
- Find all combinations of items that occur together
- They might be interesting (e.g., in placement of items in a store)
- Frequent itemsets are only positive combinations (we do not report combinations that do not occur frequently together)
- Frequent itemsets aims at providing a summary for the data

Finding frequent sets

- **Task:** Given a transaction database **D** and a **minsup** threshold find all frequent itemsets and the frequency of each set in this collection

- **Stated differently:** Count the number of times combinations of attributes occur in the data. If the count of a combination is above **minsup** report it.

How many itemsets are there?

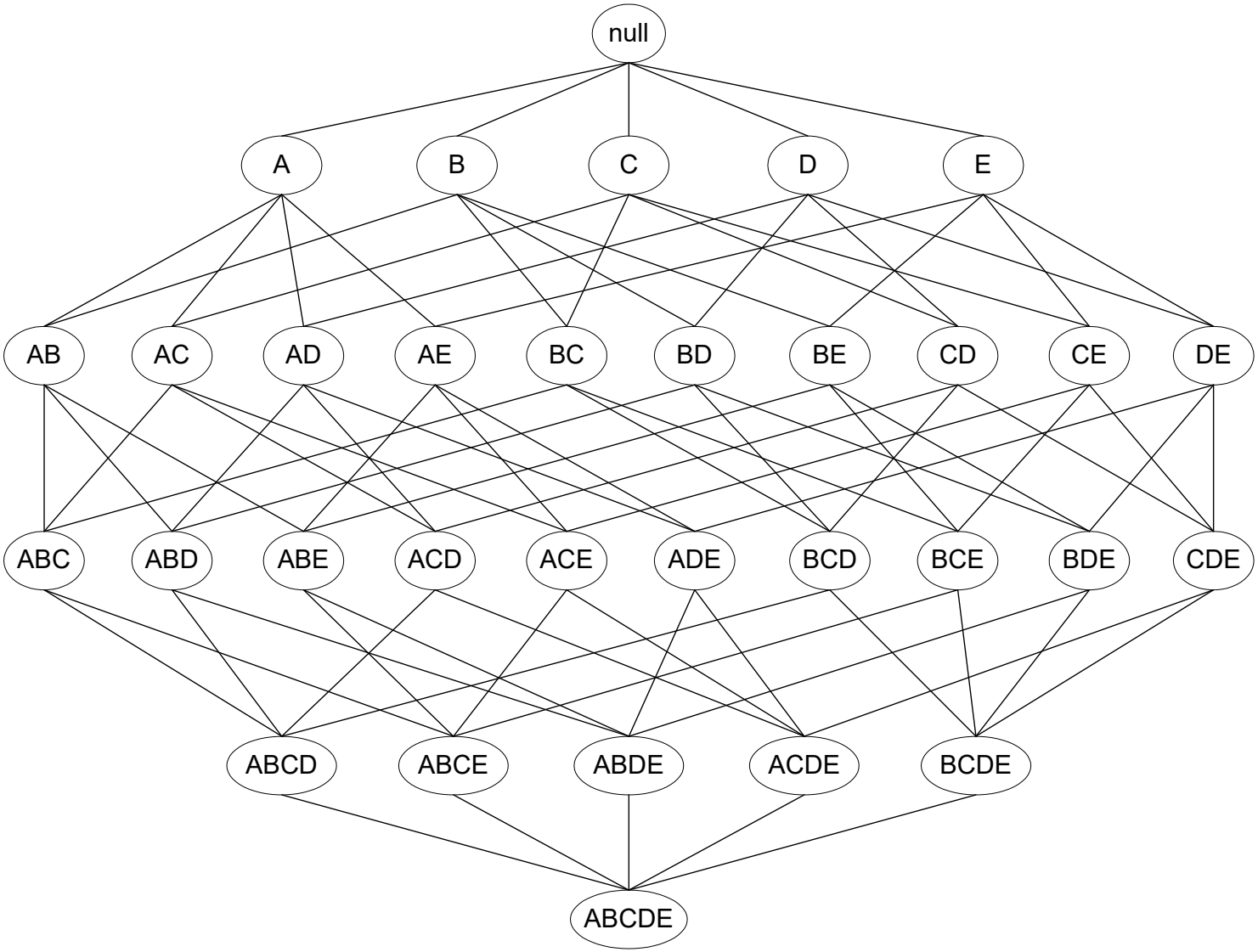


Given d items, there are 2^d possible itemsets

When is the task sensible and feasible?

- If **minsup = 0**, then all subsets of ***I*** will be frequent and thus the size of the collection will be very large
- This summary is very large (maybe larger than the original input) and thus not interesting
- The task of finding all frequent sets is interesting typically only for relatively large values of **minsup**

A simple algorithm for finding all frequent itemsets ??



Brute-force algorithm for finding all frequent itemsets?

- Generate all possible itemsets (lattice of itemsets)
  - Start with 1-itemsets, 2-itemsets,...,d-itemsets
- Compute the frequency of each itemset from the data
  - Count in how many transactions each itemset occurs
- If the support of an itemset is above **minsup** report it as a frequent itemset

Brute-force approach for finding all frequent itemsets

- Complexity?
  - Match every candidate against each transaction
  - For **M** candidates and **N** transactions, the complexity is~ **O(NM)** => Expensive since $M = 2^d$!!!

Reduce the number of candidates

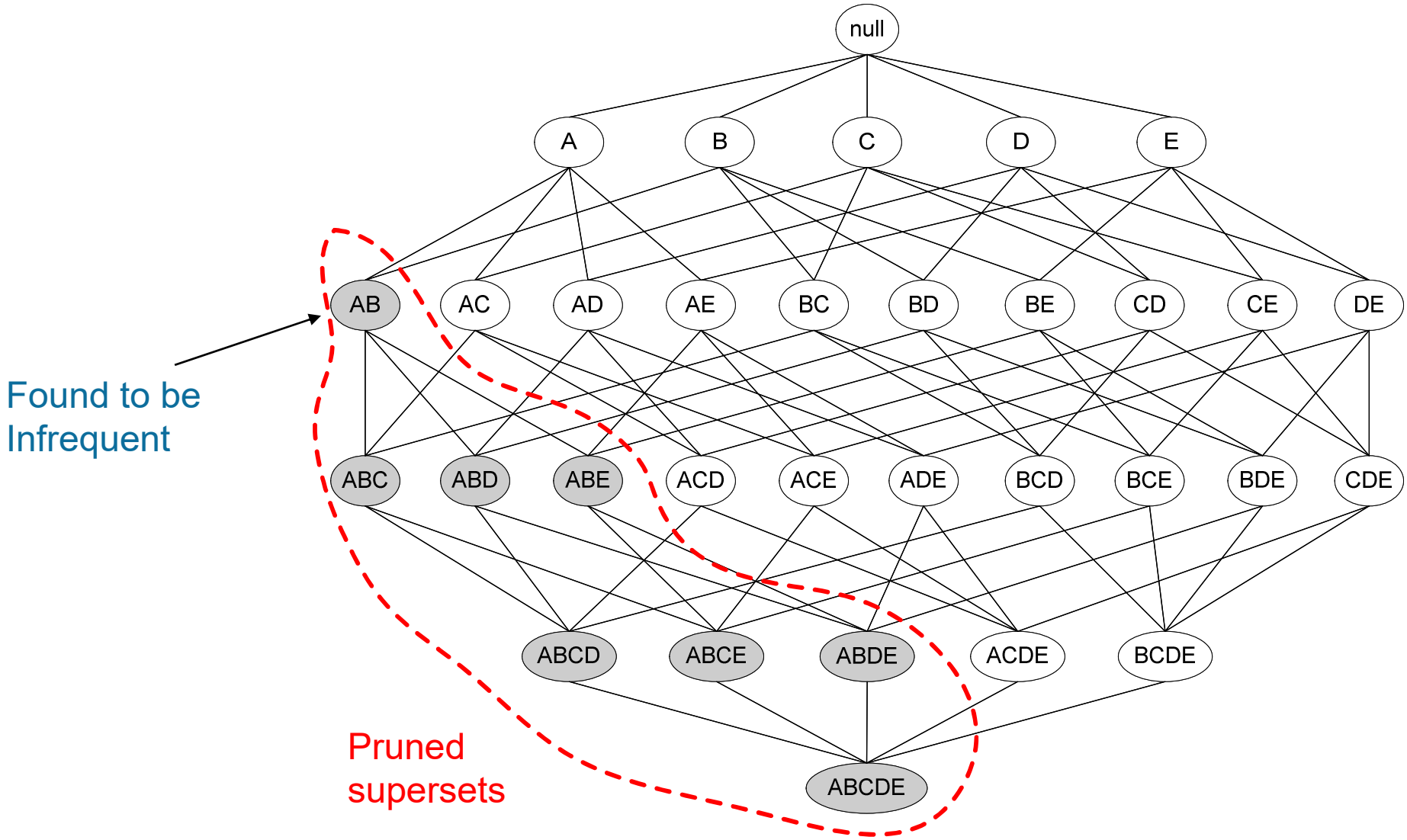
- Apriori principle (Main observation):
  - **If an itemset is frequent, then all of its subsets must also be frequent**

- Apriori principle holds due to the following property of the support measure:

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

  - The support of an itemset ***never exceeds*** the support of its subsets
  - This is known as the ***anti-monotone*** property of support

Illustrating the Apriori principle



Apriori principle (cont.)

TID	*Items*
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

s(Bread) > s(Bread, Butter)
s(Milk) > s(Bread, Milk)
s(Cookies, Butter) > s(Cookies, Butter, Coke)

An Example

- The table of transactions together with the minsup-count, are the inputs for the problem.
- The goal is to discover the strong frequent itemsets.

TID	*Items*
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

minsup-count = 3

Illustrating the Apriori principle

minsup = 3/5

Item	Count
Bread	4
Coke	2
Milk	4
Butter	3
Cookies	4
Eggs	1

Items (1-itemsets)

Itemset	Count
{Bread,Milk}	3
{Bread,Butter}	2
{Bread,Cookies}	3
{Milk,Butter}	2
{Milk,Cookies}	3
{Butter,Cookies}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)

Triplets (3-itemsets)

Itemset	Count
{Bread,Milk,Cookies}	3

Exploiting the Apriori principle

1. Find frequent 1-items and put them to L_k ($k=1$)
2. Use L_k to generate a collection of *candidate* itemsets C_{k+1} with size ($k+1$)
3. Scan the database to find which itemsets in C_{k+1} are frequent and put them into L_{k+1}
4. If L_{k+1} is not empty
   - $k=k+1$
   - Goto step 2

R. Agrawal, R. Srikant: "Fast Algorithms for Mining Association Rules",
Proc. of the 20th Int'l Conference on Very Large Databases, 1994.

The Apriori algorithm

C_k: Candidate itemsets of size k

L_k : frequent itemsets of size k

L_1 = {frequent 1-itemsets};

for (k = 2; L_k !=∅; k++)

  C_{k+1} = GenerateCandidates(L_k)

  for each transaction ***t*** in database do

    increment count of candidates in C_{k+1} that are contained in **t**

  endfor

  L_{k+1} = candidates in C_{k+1} with support ≥ ***minsup***

endfor

return $\cup_k L_k$**;**

Discussion of the Apriori algorithm

- Much faster than the Brute-force algorithm
  - It avoids checking all elements in the lattice
- The running time is in the worst case **$O(2^d)$**
  - Pruning really prunes in practice
- It makes multiple passes over the dataset
  - One pass for every level **k**
- Multiple passes over the dataset is inefficient when we have thousands of candidates and millions of transactions

Lecture outline

- **Task 1:** Methods for finding all frequent itemsets efficiently

- **Task 2:** **Methods for finding association rules efficiently**

Definition: Association Rule

Let **D** be database of transactions

– e.g.:

Transaction ID	Items
2000	A, B, C
1000	A, C
4000	A, D
5000	B, E, F

- Let ***I*** be the set of items that appear in the database, e.g., ***I={A,B,C,D,E,F}***
- A **rule** is defined by **X → Y**, where **X⊂*I***, **Y⊂*I***, and **X∩Y=∅**
  – e.g.: **{B,C} → {A}** is a rule

Definition: Association Rule

- **Association Rule**
  - An implication expression of the form **X** → **Y**, where **X** and **Y** are non-overlapping itemsets
  - Example:
    {Milk, Cookies} → ***{Butter}***

TID	Items
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

- **Rule Evaluation Metrics**
  - **Support (s)**
    - Fraction of transactions that contain both **X** and **Y**
  - **Confidence (c)**
    - Measures how often items in **Y** appear in transactions that contain **X**

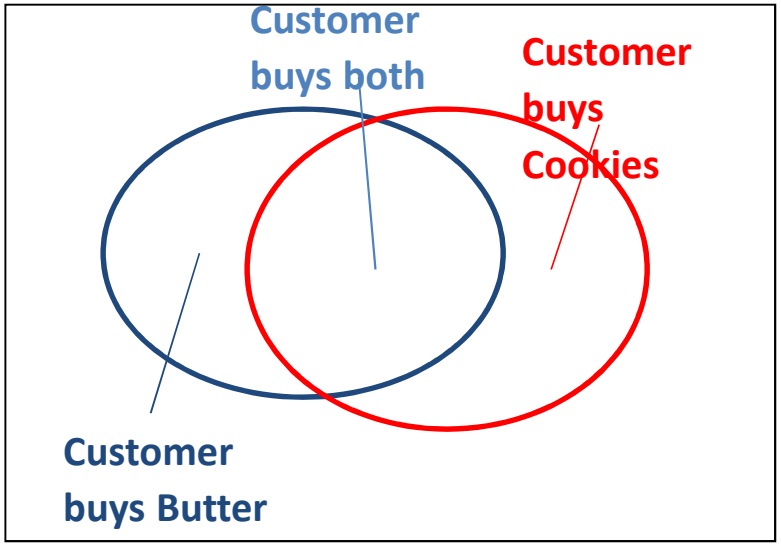
Example:

$$\{\text{Milk}, \text{Cookies}\} \rightarrow \text{Butter}$$

$$s = \frac{\sigma(\text{Milk}, \text{Cookies}, \text{Butter})}{|\text{T}|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk}, \text{Cookies}, \text{Butter})}{\sigma(\text{Milk}, \text{Cookies})} = \frac{2}{3} = 0.67$$

Rule Measures: Support and Confidence



Find all the rules $X \rightarrow Y$ with minimum confidence and support

- support, *s*, probability that a transaction contains $\{X \cup Y\}$
- confidence, *c*, ***conditional probability*** that a transaction having **X** also contains **Y**

TID	Items
100	A,B,C
200	A,C
300	A,D
400	B,E,F

Let minimum support 50%, and minimum confidence 50%, we have

- $A \rightarrow C$ (50%, 66.6%)
- $C \rightarrow A$ (50%, 100%)

Example

TID	date	items bought
100	10/10/99	{F,A,D,B}
200	15/10/99	{D,A,C,E,B}
300	19/10/99	{C,A,B,E}
400	20/10/99	{B,A,D}

What is the ***support*** and ***confidence*** of the rule: {B,D} → {A}

- Support:
  - percentage of tuples that contain {A,B,D} = 75%
- Confidence:

$$\frac{\text{number of tuples that contain } \{A,B,D\}}{\text{number of tuples that contain } \{B,D\}} = 100\%$$

Association-rule mining task

- Given a set of transactions **D**, the goal of association rule mining is to find **all** rules having
  - support ≥ ***minsup*** threshold
  - confidence ≥ ***minconf*** threshold

Mining Association Rules

TID	Items
1	Bread, Milk
2	Bread, Cookies, Butter, Eggs
3	Milk, Cookies, Butter, Coke
4	Bread, Milk, Cookies, Butter
5	Bread, Milk, Cookies, Coke

Example of Rules:

{Milk,Cookies} → {Butter} (s=0.4, c=0.67)
{Milk,Butter} → {Cookies} (s=0.4, c=1.0)
{Cookies,Butter} → {Milk} (s=0.4, c=0.67)
{Butter} → {Milk,Cookies} (s=0.4, c=0.67)
{Cookies} → {Milk,Butter} (s=0.4, c=0.5)
{Milk} → {Cookies,Butter} (s=0.4, c=0.5)

Observations:

- All the above rules are binary partitions of the same itemset: {Milk, Cookies, Butter}
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple the support and confidence requirements

Mining Association Rules

- Two-step approach:
  - Frequent Itemset Generation
    - Generate all itemsets whose support $\geq$ minsup
  - Rule Generation
    - Generate high confidence rules from each frequent itemset, where each rule is a binary partition of a frequent itemset

Efficient rule generation

- How to efficiently generate rules from frequent itemsets?
  - In general, confidence does not have an anti-monotone property

    c(ABC →D) can be larger or smaller than **c(AB →D)**

  - ***But confidence of rules generated from the same itemset has an anti-monotone property***
  - Example: X = {A,B,C,D}:

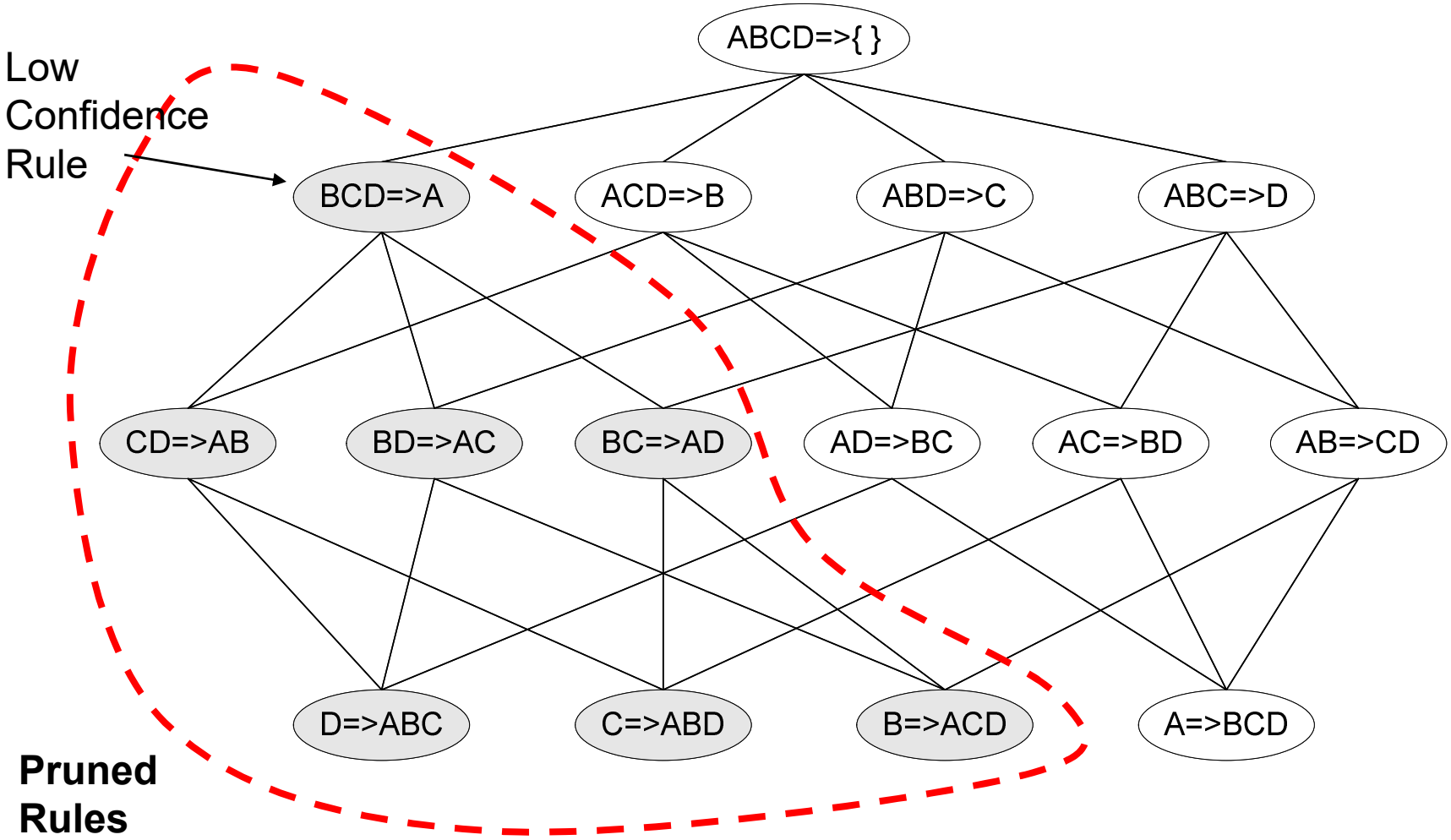
$$c(ABC \rightarrow D) \geq c(AB \rightarrow CD) \geq c(A \rightarrow BCD)$$

  - **Why?**

Confidence is anti-monotone w.r.t. number of items on the RHS of the rule.

Rule Generation for Apriori Algorithm

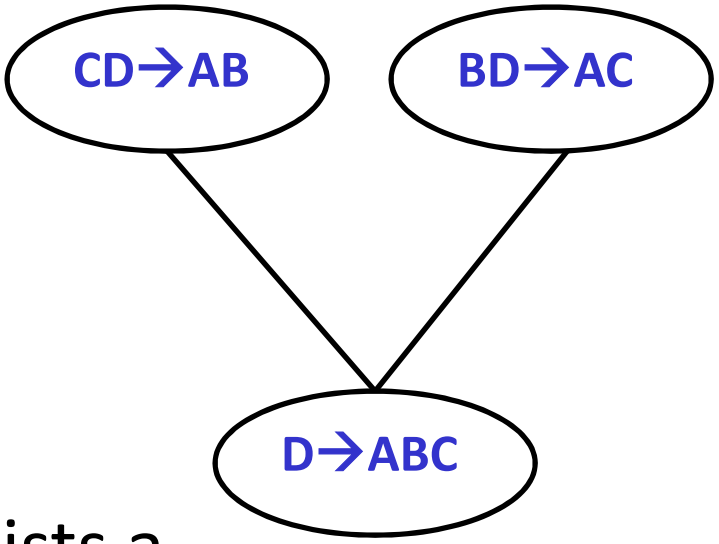
Lattice of rules



Apriori algorithm for rule generation

- Candidate rule is generated by merging two rules that share the same prefix in the rule consequent

- **join(CD→AB, BD—>AC)** would produce the candidate rule **D →ABC**



- **Prune** rule **D→ABC** if there exists a subset (e.g., **AD→BC**) that does not have high confidence

Strong Association Rules - summary

- Let I be the set of items that appear in the transaction set D, e.g., $\mathrm{I} = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}\}$. A rule $\mathrm{X} \rightarrow \mathrm{Y}$ holds in the set D with support s, where s is the percentage of transactions in D that contain both X and Y, where $\mathrm{X} \subset \mathrm{I}$ and $\mathrm{Y} \subset \mathrm{I}$, and $\mathrm{X} \cap \mathrm{Y} = \emptyset$. (note that X and Y are non-empty sets).

- The rule $\mathrm{X} \rightarrow \mathrm{Y}$ has confidence c in the transaction set D, where c is the percentage of transactions in D containing X that also contain Y.

$$\mathrm{suppor}(\mathrm{X} \rightarrow \mathrm{Y}) = \mathrm{P}(\mathrm{X} \cup \mathrm{Y})$$
$$\mathrm{confidence}(\mathrm{X} \rightarrow \mathrm{Y}) = \mathrm{P}(\mathrm{Y} \mid \mathrm{X})$$

- Rules that satisfy both a minimum support threshold (minsup) and a minimum confidence (minconf) are called **strong**.

Temporal Association Rules – an idea from a paper

In recent years, the data mining community has observed remarkable progress in discovering association rules from transactional data. The notion of association rules was first proposed by Agrawal et al. [10] to capture the co-occurrence of items in transaction events. Because a transaction typically includes a timestamp as well as the transaction items, an interesting extension to the concept of association rule is to augment it with the time dimension. This extension suggests that we may discover different rules when different time intervals are considered. As a result, some rules may hold during some time intervals, while not during the others. When we consider the association rules augmented by their temporal intervals, we refer to them as temporal association rules [11].

---

[10] R. Agrawal, T. Imielinski, and A. Swami, "Mining Associations between Sets of Items in Large Databases", ACM SIGMOD Int'l Conf. on Management of Data, Washington D.C., May 1993.

[11] Y. Li, P. Ning, X. S. Wang, and S. Jajodia, "Discovering Calendar-Based Temporal Association Rules," Data & Knowledge Engineering, vol 44, no. 2, pp. 193-218, 2003.

FP-Growth

- Frequent Pattern Growth

- Unlike the Apriori algorithm which suffers from high computational cost due to candidate generation and multiple database scans, FP-Growth avoids these inefficiencies by **compressing the data into an FP-Tree** and extracts patterns directly from it.

https://www.geeksforgeeks.org/

FP-Growth Steps

1. Data Compression into an FP-Tree
2. Mining the Tree
3. Generating Patterns

Transaction ID	Items
T1	{E,K,M,N,O,Y}
T2	{D,E,K,N,O,Y}
T3	{A,E,K,M}
T4	{K,M,Y}
T5	{C,E,I,K,O,O}

Frequency

- The frequency of each individual item is computed.

- Let minimum support be 3.
- These elements are stored in descending order of their respective frequencies.

$$L = \{K : 5, E : 4, M : 3, O : 4, Y : 3\}$$

Item	Frequency
A	1
C	1
D	1
E	4
I	1
K	5

Item	Frequency
M	3
N	2
O	4
U	1
Y	3

Ordered-Item Set

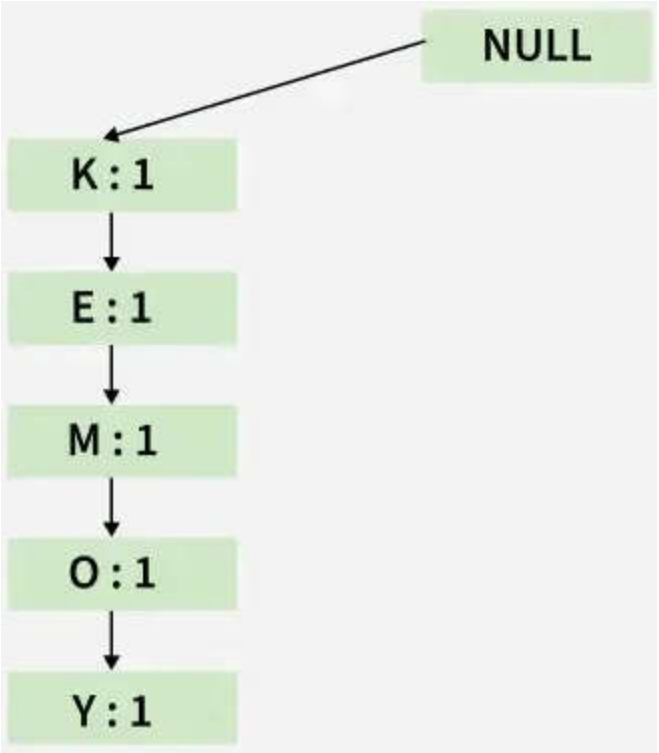
- This is done by iterating the Frequent Pattern set and checking if the current item is contained in the transaction in question.
- If the current item is contained the item is inserted in the Ordered-Item set for the current transaction.

Transaction ID	Items	Ordered-Item-Set
T1	{E,K,M,N,O,Y}	{K,E,M,O,Y}
T2	{D,E,K,N,O,Y}	{K,E,O,Y}
T3	{A,E,K,M}	{K,E,M}
T4	{C,K,M,U,Y}	{K,M,Y}
T5	{C,E,I,K,O,O}	{K,E,O}

Tree Data Structure

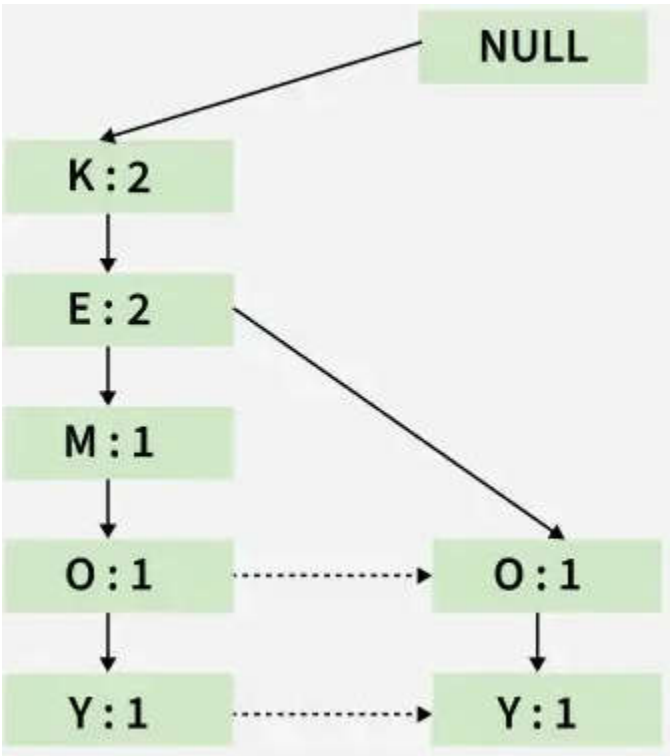
- All the Ordered-Item sets are inserted into a Tree Data Structure.

- Inserting the set {K, E, M, O, Y}.

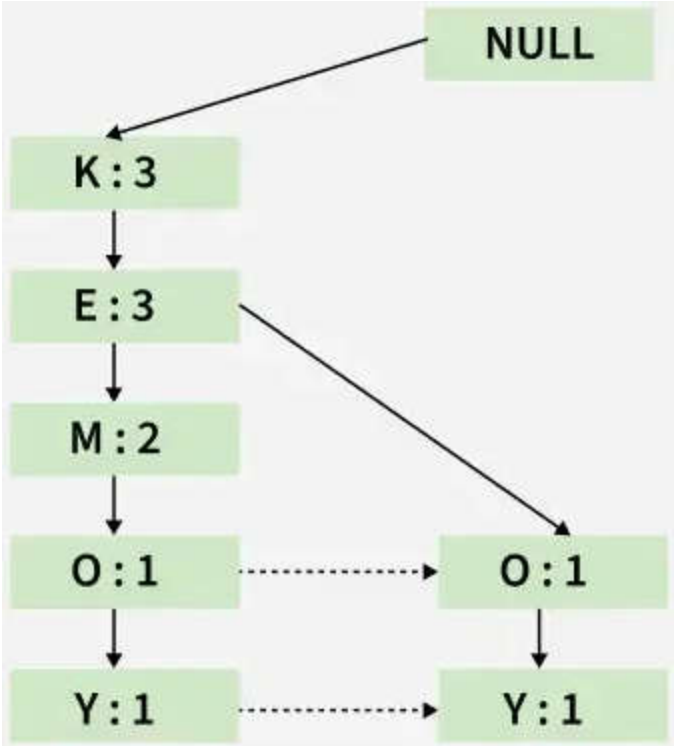


Tree Data Structure (cont.)

- Inserting the set {K, E, O, Y}

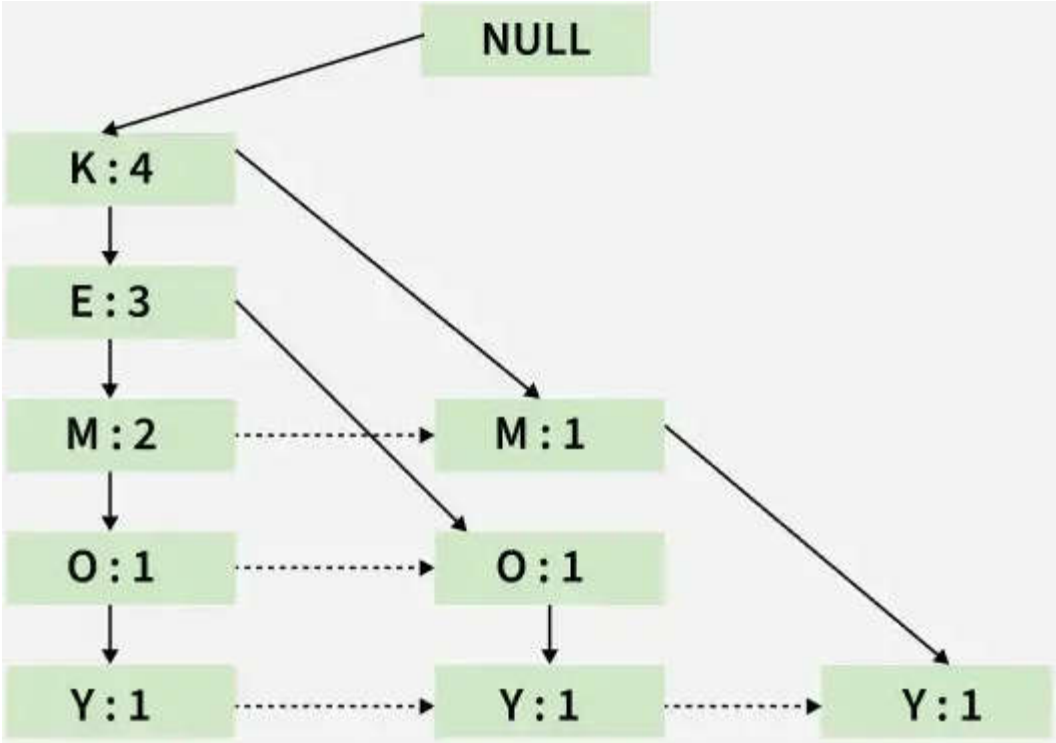


- Inserting the set {K, E, M}



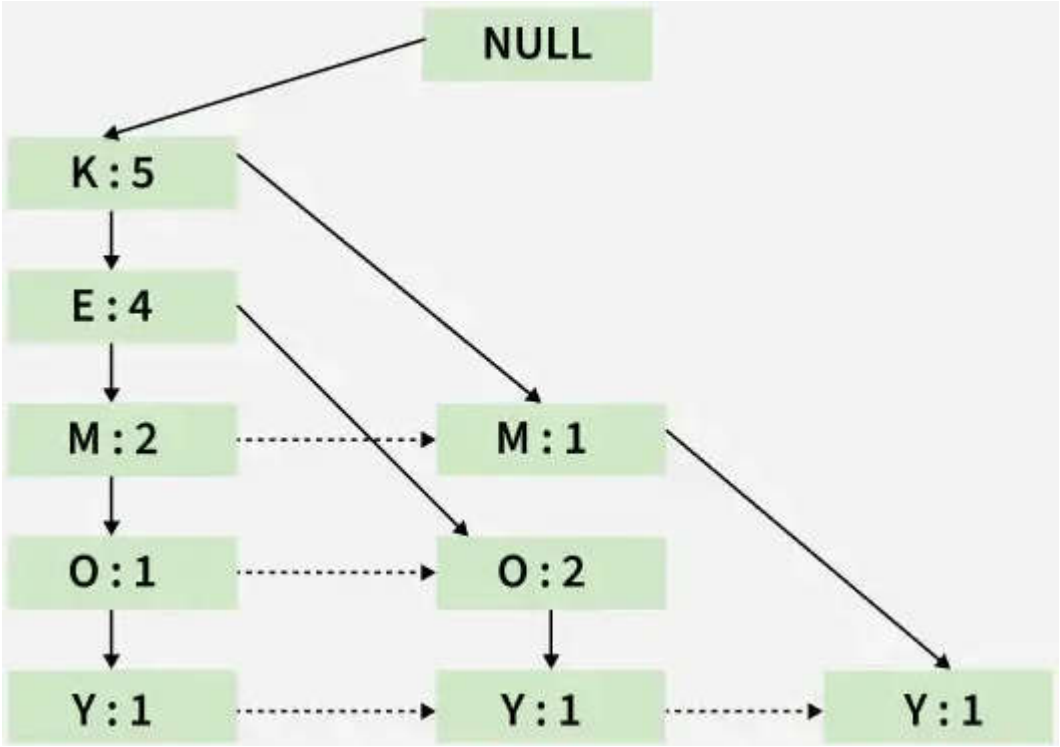
Tree Data Structure (cont.)

- Inserting the set {K, M, Y}



Tree Data Structure (cont.)

- Inserting the set {K, E, O}



Conditional Pattern Base

- The Conditional Pattern Base for each item consists of the set of prefixes of all paths in the FP-tree that lead to that item.

- The items in the below table are arranged in the ascending order of their frequencies.

Items	Conditional Pattern Base
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}
O	{{K,E,M : 1}, {K,E : 2}}
M	{{K,E : 2}, {K : 1}}
E	{K : 4}
K	

Conditional Frequent Pattern Tree

- For each item, the **Conditional Frequent Pattern Tree is built.**

- This is done by taking the set of elements that is common in all the paths in the Conditional Pattern Base of that item and calculating its support count by summing the support counts of all the paths in the Conditional Pattern Base.

Items	Conditional Pattern Base	Conditional Frequent Pattern Tree
Y	{{K,E,M,O : 1}, {K,E,O : 1}, {K,M : 1}}	{K : 3}
O	{{K,E,M : 1}, {K,E : 2}}	{K,E : 3}
M	{{K,E : 2}, {K : 1}}	{K : 3}
E	{K : 4}	{K : 4}
K		

Frequent Pattern Rule

- From the Conditional Frequent Pattern tree the **Frequent Pattern rules** are generated by pairing the items of the Conditional Frequent Pattern Tree set to the corresponding to the item.

Items	Frequent Pattern Generated
Y	{<K,Y : 3>}
O	{<K,O : 3>, <E,O : 3>, <E,K,O : 3>}
M	{<K,M : 3>}
E	{<E,K : 4>}
K	

Conclusion

- Frequent Pattern Growth (FP-Growth) algorithm improves upon the Apriori algorithm by eliminating the need for multiple database scans and reducing computational overhead.

- By using a Tree data structure and focusing on ordered-item sets it efficiently mines frequent item sets making it a **faster** and more **scalable** solution for large datasets making it useful tool for data mining.

https://www.geeksforgeeks.org/